

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 02 -

GIT E CONTROLE DE VERSÃO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	22
O QUE VOCÊ VIU NESTA AULA?	25
REFERÊNCIAS.....	26

EMANIP

O QUE VEM POR AÍ?

Imagine a frustração de perder horas de trabalho porque alguma parte do código foi sobrescrita ou porque você não consegue voltar a uma versão anterior do seu projeto de ciência de dados. Já se viu nomeando arquivos como “analise_final_v2_definitivo” para tentar controlar versões? Esse cenário comum reflete a falta de um controle de versão adequado.

Nesta seção inicial, vamos quebrar o gelo discutindo a importância do controle de versão e como o Git se tornou a solução padrão para esse problema. Você refletirá sobre os desafios de gerenciar mudanças de código ao longo do tempo e como o Git — um sistema de controle de versão distribuído — veio resolver esses desafios de forma elegante.

O Git permite registrar cada alteração no código como um commit com mensagem descritiva, construir um histórico completo do projeto e, se necessário, reverter alterações de forma segura. Isso significa que, diferente de métodos manuais (como “Track Changes” em documentos de texto ou salvar múltiplas cópias do arquivo), com o Git você pode experimentar novas ideias sem medo: se algo der errado, basta voltar a um estado anterior em poucos comandos.

Ao longo deste material, você aprenderá a inicializar repositórios Git, salvar mudanças com commits bem documentados e organizar o trabalho em branches (ramificações) para desenvolver novas funcionalidades sem interferir no código principal. Também vamos discutir como o Git garante histórico completo e oferece maneiras seguras de desfazer mudanças (por exemplo, usando `git revert` em vez de apagar histórico) para que nada seja perdido.

Ao final, você estará convencido(a) de como essas práticas elevam a qualidade do seu código e facilitam a colaboração, preparando o terreno para nos aprofundarmos em workflows de equipe e resolução de conflitos nas próximas etapas do curso.

HANDS ON

O objetivo aqui é ver, de forma simples e direta, como iniciar um repositório, registrar alterações e trabalhar com branches. Suponha que você esteja começando um novo projeto chamado AnaliseClima: ele pode conter um script Python que lê dados meteorológicos e calcula estatísticas simples. Iniciaremos executando comandos Git no terminal (Linux/macOS ou Windows via Git Bash) para configurar o controle de versão localmente.

Em seguida, faremos modificações no código e simularemos a evolução do projeto através de commits sequenciais. Lembre-se de que todos os comandos mostrados estão disponíveis na documentação oficial do Git e em inúmeros tutoriais – se necessário, **revisite este conteúdo e as videoaulas associadas sempre que sentir necessidade de reforçar esses passos!**

Primeiro, vamos criar a pasta do projeto e inicializar um repositório Git dentro dela:

```
$ mkdir AnaliseClima && cd AnaliseClima
$ git init
Initialized empty Git repository in /AnaliseClima/.git/
$ git status
On branch main
No commits yet
```

Código-fonte 1 – Comandos para criar diretório do projeto e inicializar um repositório Git (Bash)
Fonte: Elaborado pelo autor (2025)

O comando git init cria um novo repositório local (note que estamos na branch padrão “main” e ainda não há commits). Em seguida, vamos adicionar um código em Python ao repositório. Crie um arquivo analise.py com o conteúdo a seguir, que calcula a média de temperaturas de uma lista de valores:

```
# analise.py - Versão 1
temperaturas = [22.4, 21.8, 23.1, 20.0, 19.5]
media = sum(temperaturas) / len(temperaturas)
print(f"Média das temperaturas = {media:.2f}°C")
```

Código-fonte 2 – Script Python simples para cálculo de temperatura média (versão inicial)
Fonte: Elaborado pelo autor (2025).

Agora vamos versionar esse código. Primeiro, precisamos informar ao Git para acompanhar (git add) o arquivo, depois criar o primeiro commit com git commit:

```
$ git add analise.py
$ git commit -m "feat: calcula média de temperatura inicial"
[main (root-commit) ea8321d] feat: calcula média de temperatura inicial
 1 file changed, 5 insertions(+)
 create mode 100644 analise.py
$ git status
On branch main
nothing to commit, working tree clean
```

Código-fonte 3 – Comandos Git para adicionar o arquivo e criar um commit inicial com mensagem descritiva (Bash)

Fonte: Elaborado pelo autor (2025).

No comando de commit anterior, usamos uma mensagem seguindo uma convenção simples: começando com feat: para indicar uma nova funcionalidade. Boas práticas recomendam usar mensagens claras e padronizadas – por exemplo, o padrão Conventional Commits sugere prefixos como feat (funcionalidade), fix (correção) etc. e limitar a linha de assunto a 50 caracteres. Isso torna o histórico mais legível e útil. Note que o Git identificou que 1 arquivo foi alterado/adicionado (“1 file changed, 5 insertions”) e agora o status indica que não há mudanças pendentes (working tree clean).

SAIBA MAIS

Vamos nos aprofundar nos conceitos e melhores práticas de engenharia de software por trás do controle de versão com Git. Nesta seção, adotaremos um tom didático e denso, trazendo aspectos teóricos e práticos. Lembre-se: apesar do foco técnico, você pode consultar as videoaulas e o material de apoio sempre que precisar reforçar o entendimento, além de aproveitar para pausar e refletir sobre cada conceito introduzido aqui.

Em essência, um sistema de controle de versão é uma ferramenta que registra alterações em arquivos ao longo do tempo, permitindo que você retorne a versões específicas mais tarde. Isso é especialmente valioso em projetos de ciência de dados, em que experimentamos abordagens diversas em código e queremos comparar resultados de diferentes versões de algoritmos ou análises.

O Git tornou-se o sistema de controle de versão dominante, utilizado por cerca de 93% de especialistas em desenvolvimento em todo o mundo de acordo com pesquisas de 2023. Os motivos para essa ampla adoção incluem flexibilidade, velocidade e modelo distribuído de funcionamento. Modelo distribuído significa que cada colaborador(a) possui, em seu computador, uma cópia completa do repositório (todos os arquivos e todo o histórico de commits).

Isso difere dos antigos sistemas centralizados (como SVN), nos quais era necessário estar conectado a um servidor para cada operação – no Git, você pode fazer commits e navegar pelo histórico localmente, mesmo sem internet. Além disso, operações comuns (commits, merges) tendem a ser rápidas, pois ocorrem localmente; o uso de estruturas eficientes (como algoritmos de diffs e compressão de dados) faz do Git uma ferramenta capaz de lidar com projetos grandes sem exaurir recursos de CPU ou memória na maior parte dos casos.

Uma das principais recomendações de engenharia de software é fazer commits pequenos e frequentes, ou seja, commits atômicos. Um commit atômico representa uma única alteração lógica no código – por exemplo, corrigir um bug específico ou adicionar uma função em separado.

Esse hábito traz vários benefícios: facilita identificar quando um problema foi introduzido (cada mudança isolada pode ser analisada separadamente) e torna

viável reverter apenas aquela alteração, se necessário, sem desfazer outras melhorias feitas no mesmo período. Em contrapartida, commits “monolíticos” (com muitas alterações de naturezas distintas de uma vez) dificultam a vida de quem analisa o histórico, torna complicado localizar bugs e praticamente inviabiliza revertê-los parcialmente. Uma boa metáfora é pensar nos commits como etapas de um experimento científico: documentamos cada passo do processo ao invés de apenas anotar o resultado final.

Tão importante quanto o escopo dos commits é a mensagem de commit que os acompanha. A mensagem deve descrever claramente o que foi alterado e por quê. No contexto de equipes (e mesmo para você, se revisitar o código meses depois), a mensagem de commit funciona como uma mini documentação do histórico do projeto. Considere a diferença entre uma mensagem vaga como “Atualiza código” e outra descritiva como “fix: corrige cálculo de média móvel no módulo de previsão – resolves #23”. A segunda explica exatamente a natureza da mudança e ainda referencia um problema rastreável (no caso, “issue #23”).

Boas mensagens poupam horas de depuração no futuro, pois permitem navegar no histórico e entender rapidamente cada decisão tomada. Para atingir esse nível de qualidade, várias equipes adotam convenções de formatação. Uma dica clássica é a regra dos 50/72 caracteres, que sugere limitar a linha de assunto do commit a 50 caracteres e, se necessário, incluir um corpo explicativo com quebras de linha em 72 caracteres. Isso garante que as mensagens fiquem legíveis em diversas ferramentas (como no git log no terminal ou em telas do GitHub) sem truncar.

Outras convenções populares incluem começar a mensagem com um verbo no imperativo (“Adiciona função X”, “Remove Y”, “Corrige Z”), o que enfatiza a ação efetuada pelo commit. O importante é ser consistente: definir e seguir um estilo padrão de mensagens dentro do time melhora a legibilidade do histórico e passa profissionalismo. De fato, estudos de engenharia de software mostram que commits bem documentados tornam o processo de code review mais rápido e eficaz, além de evitar retrabalho devido à falta de contexto.

O Git se destaca por suas ramificações (branches) leves e poderosas. Criar uma branch no Git é uma operação quase instantânea e sem custo significativo de espaço, diferentemente de sistemas legados em que branches eram pesados e

pouco usados. Esse design incentiva a utilização de branches para isolar o desenvolvimento de novas funcionalidades, experimentos ou correções.

Mesmo em projetos pessoais de ciência de dados, vale a pena utilizar branches ao testar uma ideia radical: você cria uma branch a partir do estado estável do projeto (por exemplo, experimento-redes-neurais a partir da main) e pode codificar livremente. Se a ideia não der certo, basta descartar a branch; se der certo, o Git permite mesclar (merge) as mudanças de volta à branch principal. Essa abordagem traz segurança psicológica – você explora mais porque sabe que sua base está preservada – e mantém o histórico organizado, já que cada branch pode contar a “história” de um desenvolvimento separado até o momento de integração.

Enquanto está trabalhando em uma branch separada, é importante também integrar regularmente as mudanças da branch principal (main) no seu branch para evitar divergências muito grandes. Suponha que você criou a branch melhoria-A e, enquanto trabalha nela, outra pessoa fez commits importantes na main. Se você ignorar essas atualizações por muito tempo, corre o risco de enfrentar conflitos extensos quando finalmente for mesclar.

Por isso, práticas como “atualizar o branch de feature” com merge ou rebase frequentes a partir da main são recomendadas. Merge e rebase são duas estratégias de integração: o merge cria um commit de junção, como vimos no hands on, mantendo o histórico divergente, enquanto o rebase “reaplica” seus commits por cima da nova base da main, criando um histórico linear. Cada estratégia tem prós e contras – o importante é não deixar as branches “envelhecerem” afastadas da linha principal. Em projetos de ciência de dados, em que o código muda rapidamente durante análises exploratórias, essa recomendação ajuda a minimizar surpresas ao consolidar resultados.

Um dos mandamentos ao usar controle de versão é não apagar histórico compartilhado. Alterar ou perder histórico elimina exatamente os benefícios do versionamento (auditoria, reprodutibilidade e “linha do tempo” do projeto). Por isso, o Git oferece mecanismos para desfazer enganos de maneira segura: o comando git revert é o principal. Diferentemente de git reset (que volta o estado do repositório e pode remover commits do histórico), o revert cria um commit inverso que anula as mudanças de um commit específico, sem excluir nada do histórico. Pense no revert

como um CTRL+Z profissional, que deixa um “rastros” do desfazer, em vez de simplesmente apagar.

Por exemplo, se em um commit você adicionou uma linha incorreta de código, o revert fará outro commit removendo essa linha — o histórico passa a mostrar ambos: a adição e a remoção, com as respectivas mensagens explicando cada um. Isso é vital em ambientes de equipe: tentar eliminar completamente um commit que já foi enviado ao repositório central (e possivelmente puxado por outros desenvolvedores e desenvolvedoras) causa um desalinhamento chamado de history rewrite, gerando conflitos para todos. Em muitos fluxos, reescrever a história compartilhada é um tabu.

Dica: utilize `git reset` apenas para corrigir commits antes de enviá-los ao servidor remoto (por exemplo, você fez um commit local mas percebeu um erro de digitação na mensagem ou esqueceu de incluir um arquivo — aí um `git reset --soft` pode “desfazer” o commit localmente, permitindo refazê-lo corretamente). Mas se o commit já foi publicado e sincronizado com a equipe, opte pelo `revert` para corrigir quaisquer problemas. Essa distinção reforça a ideia de histórico imutável no repositório principal, garantindo confiança: todos sabem que os commits uma vez integrados não vão simplesmente desaparecer ou mudar de identidade (hash).

Vale comentar que o Git armazena os dados de forma eficiente usando um modelo de snapshot (instantâneo) e não diferenças lineares como algumas pessoas podem pensar. Cada commit no Git guarda um snapshot do conteúdo de todos os arquivos versionados naquele momento (na verdade, arquivos inalterados entre commits são referenciados pelo mesmo objeto interno, de forma que não se duplica dados; além disso, as packfiles comprimem objetos para economizar espaço).

Esse modelo baseado em hash (cada commit tem um identificador SHA-1/SHA-256 derivado de seu conteúdo e históricos anteriores) garante integridade — qualquer modificação acidental ou maliciosa em um arquivo será detectada porque seu hash não corresponderá ao do commit registrado.

Ou seja, o Git trata a história como sagrada: se algo difere, não é o mesmo commit. Esse rigor matemático por trás do Git dá tranquilidade quanto à fidedignidade do histórico e conecta com conceitos de computação como funções de hash criptográficas e grafos acíclicos dirigidos (o histórico Git é essencialmente um

grafo acíclico onde os nós são commits e as arestas referenciam commits antecessores).

Embora o Git tenha surgido no contexto de desenvolvimento de software (foi criado por Linus Torvalds em 2005 para ajudar a coordenar milhares de contribuições ao kernel Linux), ele é aplicável – com alguns cuidados – ao fluxo de trabalho de cientistas de dados. Por exemplo, datasets grandes e arquivos binários de modelo não versionam bem com Git, que foi pensado principalmente para código-texto.

Isso significa que incluir arquivos CSV enormes ou modelos de rede neural pesando gigabytes no repositório Git pode tornar operações lentas e o histórico pesado. Para contornar esse limite, surgiram extensões como Git LFS (Large File Storage) e ferramentas como o DVC (Data Version Control), que permitem rastrear metadados desses arquivos nos commits e armazenar o conteúdo pesado externamente.

Assim, você consegue manter a reprodutibilidade (sabendo qual versão dos dados foi usada em cada commit) sem sobrecarregar o Git com os dados em si. Em termos de engenharia, é uma separação de responsabilidades: Git para código e pequenos arquivos de configuração; DVC ou outros para dados brutos e artefatos volumosos de ML.

Outro aspecto relevante é a reprodutibilidade e colaboração. Em ciência de dados, frequentemente trabalhamos em pequenos times ou em pesquisas acadêmicas compartilhando código. Utilizar Git e GitHub traz para esse domínio a mesma confiança e produtividade que desenvolvedores(as) de software já desfrutam.

Por exemplo, no Pew Research Center, a adoção do Git/GitHub em projetos de análise de dados trouxe consistência entre projetos, transparência e colaboração mais efetiva, ao ponto de nenhum membro tomar decisões isoladamente sem passar por revisão por pares via pull requests. O código torna-se mais legível e confiável, e novos integrantes conseguem entender o caminho das análises lendo o histórico de commits e pull requests (como se fosse um diário do projeto).

Além disso, práticas de controle de versão suportam a ciência reproduzível – um colega ou revisor pode pegar a versão exata do código e dados usados para

gerar uma conclusão científica e verificar os resultados. Nesse sentido, a engenharia de software e a ciência de dados se encontram: controlar versões de forma disciplinada é fundamental para confiarmos em experimentos e modelos no longo prazo.

Por fim, do ponto de vista matemático e computacional, o Git nos expõe a conceitos interessantes: teoria de grafos (o commit DAG – Directed Acyclic Graph – do histórico), hashing criptográfico (SHA-1) e algoritmos de diffs que encontram eficientemente mudanças entre versões de texto. Até mesmo estatística aparece: é possível extrair métricas do repositório para avaliar produtividade (número de commits por semana), cálculo de bus factor (quantas pessoas concentram a maioria das alterações) ou aplicar análise de redes para ver quais arquivos tendem a mudar juntos.

Essas métricas às vezes são chamadas de analítica de repositório e podem orientar decisões de gerenciamento de projeto. Contudo, cuidado: volume de commits não é sinônimo de qualidade! O valor está em usar o repositório como fonte de insights, não como competição de números.

Conforme antecipamos, existem diferentes maneiras de organizar o uso de branches e merges em um projeto com múltiplos colaboradores(as). Não há uma forma obrigatória – a escolha depende do contexto do projeto (tamanho da equipe, frequência de releases, criticidade do software). Vamos comparar três modelos conhecidos:

Git Flow: proposto por Vincent Driessen em 2010, popularizou o uso de várias branches com papéis específicos: uma branch principal (main ou master) para versões em produção, uma branch de desenvolvimento integrada (develop) onde entram as features aprovadas, branches de feature (funcionalidades) que saem de develop e retornam para ela, branches de release para preparar uma versão (saem de develop e depois de prontas são mescladas em main e develop) e branches de hotfix para correções urgentes que saem de main e depois retornam para ambos main e develop.

Esse modelo dá bastante estrutura e controle, permitindo desenvolvimentos paralelos sem quebrar o código de produção. Por exemplo, enquanto features são desenvolvidas na integração contínua de develop, uma equipe pode estar testando

uma release candidata em uma branch release/1.0. Uma vantagem do Git Flow é a clareza: sabe-se exatamente onde está o código estável e onde estão as novidades em teste. Também é útil para projetos que lançam versões discretas (ex.: software com versões X.Y.Z).

No entanto, desvantagens incluem complexidade de gerenciamento de muitas branches e deploys mais lentos (já que se espera acumular várias features antes de lançar). Em contexto de ciência de dados, o Git Flow pode ser “demais” se o projeto não segue versões formais – muitas equipes de dados preferem abordagens mais simples, mas se estiverem desenvolvendo, por exemplo, um pacote Python para uso interno, poderiam usar Git Flow durante o ciclo de desenvolvimento e lançamento.

GitHub Flow: é um workflow mais simples, adequado a entregas contínuas e muito popular em projetos ágeis e open-source modernos. Em resumo, tudo acontece em torno da branch principal (main), que sempre deve estar pronta para deploy. Cada item de trabalho (feature, melhoria ou correção) é desenvolvido em uma branch curta criada a partir da main. Assim que estiver pronto (e possivelmente até antes disso, para trabalho em progresso), é aberto um Pull Request dessa branch para a main. Depois de revisado e aprovado, é feito o merge na main – idealmente, várias vezes por dia há merges ocorrendo.

Não existem branches develop ou release: o código vai direto para a main e geralmente um processo automatizado de CI/CD pode implantar as mudanças em produção rapidamente (muitas empresas fazem deploys múltiplos por dia usando esse modelo). As vantagens do GitHub Flow são a simplicidade e a agilidade: menos branches significam menos sobrecarga de sincronização e as integrações frequentes significam menos conflitos acumulados e feedback rápido de testes.

A contraface é que isso exige maturidade em automação – sem um sólido conjunto de testes e integração contínua, pode haver risco de algo imaturo ir para produção, já que o ciclo é rápido. No contexto de dados, se pensarmos em um pipeline de dados em produção (que extrai, transforma e carrega dados periodicamente), adotar o GitHub Flow significaria que qualquer ajuste no pipeline seria desenvolvido em uma branch separada e integrada assim que testado, possivelmente indo ao ar na próxima execução pipeline.

Isso traz rapidez na melhoria, mas demanda que cientistas/engenheiros de dados mantenham testes para garantir que as mudanças não quebrem nada – felizmente, isso é uma boa prática de qualquer forma.

Trunk-Based Development (TBD): é semelhante ao GitHub Flow no sentido de ter a main (trunk) como centro, mas foca ainda mais em merges extremamente frequentes, mantendo branches de vida muito curta (horas ou poucos dias, no máximo). Muitas vezes, times trunk-based nem chegam a usar pull requests formais – em alguns casos, fazem pair programming ou mob programming para revisar em tempo real ou implementam ferramentas de merge queue que serializam automaticamente merges na main com testes.

O trunk-based é quase sinônimo de Continuous Integration estrita: todos integrando o tempo todo. Benefícios incluem throughput alto (as mudanças fluem rapidamente para o produto) e menos conflitos de longa duração, alinhando-se bem a métricas de alto desempenho (como mencionado, times trunk-based frequentemente pontuam melhor em métricas DORA).

Porém, é talvez o mais desafiador de adotar sem uma cultura forte: requer disciplina para fazer commits pequenos, recursos como feature flags para esconder funcionalidades inacabadas (já que tudo vai pra main mesmo antes de completo) e altíssima confiança em testes automatizados e monitoramento em produção. Poucas equipes de ciência de dados operam trunk-based puro, mas elementos dele aparecem quando o time opta por não ter uma branch develop e sempre trabalhar próximo da main.

Uma comparação resumida ajuda a fixar as diferenças.

ASPECTO	GIT FLOW (TRADICIONAL)	TRUNK-BASED (MODERNO)
Branches principais	main (produção) e develop (integração)	Apenas main (trunk)
Branches de suporte	Múltiplos: feature/, release/, hotfix/*	Apenas branches curtas de feature

Duração das branches	Podem ser longas (semanas até release)	Curtíssimas (horas/dias)
Release (deploy)	Agrupado por versões, menos frequente	Contínuo, cada merge pode ser deployável
Controle de qualidade	Estrutura rígida + testes em release branches	Testes automatizados a cada commit (CI)
Risco de conflito	Pode acumular; mitigado via branch develop	Mínimo – integração constante
Quando usar	Equipes/grupos com lançamentos pontuais, necessidade de estabilidade pré-release, ou compliance/regulação exigindo processos formais.	Equipes ágeis com entrega contínua, foco em rapidez e capacidade de re-vert rápido se precisar

Tabela 1 – Comparação de estratégias de branching (Git Flow vs. Trunk-Based)
Fonte: Elaborado pelo autor (2025), com base em CapGov (2025) e Mergify (2025).

Como você pode notar, nenhum modelo é intrinsecamente melhor – eles servem a propósitos distintos. Projetos open-source no GitHub geralmente seguem o GitHub Flow; grandes empresas tech tendem ao Trunk-Based; software com versões do tipo “v1.0, v2.0” ou ambientes muito críticos podem ficar com Git Flow. E vale mencionar: existem variações híbridas. Por exemplo, o GitLab Flow integra ideias do GitHub Flow com uso de branches de ambiente (ex.: main espelhando produção,

staging espelhando homologação). O importante é o time adotar um modelo e segui-lo de forma consistente, comunicando claramente nos docs do projeto.

Independentemente do workflow escolhido, revisão de código é uma prática universal e altamente recomendada. No GitHub, isso acontece via Pull Requests, mas em outras plataformas ou em Git pura poderia ser via patches e análise manual. Vamos focar no PR do GitHub, pois é o mais comum. Quais são as boas práticas para PRs e code reviews?

- **Tamanho e escopo:** PRs devem ser razoavelmente pequenos e focados. Um PR ideal trata de uma única feature ou correção e evita misturar mudanças não relacionadas. Por exemplo, não inclua refatorações de formatação de código em um PR que implementa uma funcionalidade nova – isso dificulta a revisão. PRs pequenos tendem a ser revisados mais rapidamente (menos carga cognitiva para o revisor). Um estudo interno no Microsoft Azure DevOps, por exemplo, mostrou que PRs revisados em até 24h tinham, em média, < 10 arquivos modificados – acima disso o tempo de revisão subia bastante. Portanto, “Faça PRs pequenos e com propósito claro” é o mantra.
- **Descrição clara:** ao abrir um PR, preencha a descrição com detalhes úteis: o que foi feito e por que. Se o PR resolve uma issue ou tarefa, referencie (“Closes #123”). Incluir screenshots ou outputs, se pertinente (por exemplo, se a mudança afeta um gráfico ou relatório, colocar uma imagem do resultado esperado ajuda o revisor). Lembre-se que o PR serve também de documentação futura: meses depois, alguém lendo o histórico deve entender o contexto daquela mudança.
- **Participação do time:** defina colegas como revisores no PR. No GitHub, você pode marcar usuários ou equipes. Em times de ciência de dados, às vezes todos revisam de todos ou alternam pares. O importante é ter pelo menos uma outra pessoa olhando cada PR (four eyes principle). Revisores devem fazer comentários construtivos, focando no código e não na pessoa. Exemplo: em vez de “Código confuso aqui”, prefira “Poderia explicar a razão dessa abordagem? Talvez extrair para uma função ajudaria a legibilidade.” – mantendo respeito e objetividade.

- **Ferramentas de automação na revisão:** utilize integração de linters e formatadores (como flake8, black para Python) e análise estática (SonarQube, etc.) no CI, assim problemas de estilo ou bugs óbvios são detectados automaticamente, poupando tempo do revisor humano para questões mais importantes. O GitHub PR mostra checks de CI passando ou falhando; muitos times só revisam funcionalmente o PR se os checks estiverem verdes (aprovados).
- **Tempo de resposta:** combine SLAs informais – ex.: pull requests devem ser revisados em até 1 dia útil. Assim evita-se PRs ficando abertos por muito tempo (o que trava quem abriu e também aumenta a chance de conflitos se outros trabalhos seguem). Quem abre o PR também deve estar disponível para responder a feedbacks rapidamente (é comum iterar: revisor comenta X, autor faz commit de ajuste, revisor comenta Y, etc.). Comunicação ágil via PR ou chat ajuda a resolver dúvidas sem arrastar o processo.
- **Merge e pós-merge:** após aprovação, o merge deve respeitar o modelo escolhido (se squash ou não). No GitHub, costuma-se usar squash commits se houver muitos “fix typo” e “address review” que poluem o histórico, mas isso elimina ver os commits individuais – equipes têm preferências diferentes. Seja qual for, após o merge lembre de fechar a branch (o GitHub pergunta). E a cada merge na main, se há pipeline de implantação contínua, monitore os resultados – idealmente, todo PR mergeado dispara testes e possivelmente um deploy; caso algo dê errado em produção, as ferramentas de monitoramento/devOps devem alertar para se fazer rollback (tal rollback pode ser simplesmente um novo PR revertendo o commit problemático, outra prova da utilidade do git revert).

Implementar code reviews traz ganhos significativos: estudos indicam que encontrar e corrigir defeitos em revisão de código é muito mais barato do que fazê-lo após o código em produção. Além disso, revisões promovem compartilhamento de conhecimento – um colega aprende um truque de Python que não conhecia, outro entende melhor determinada parte do sistema ao revisar o código de alguém. Para cientistas de dados, isso também vale: code review pode pegar aquele gráfico cujo

eixo estava sem rótulo ou um cálculo estatístico implementado de forma ineficiente, antes que vá para um relatório final.

Conflitos de merge são uma realidade, mas não precisam ser um bicho de sete cabeças. Vamos abordar algumas estratégias para minimizar conflitos e como agir quando eles ocorrem:

- **Comunicação e coordenação:** ferramental é ótimo, mas comunicação humana ainda é essencial. Em equipes, mantenha todos informados sobre em que estão trabalhando. Se notar que você e outra pessoa planejam editar o mesmo módulo, conversem e talvez dividam a tarefa ou alinhem a sequência (ex.: “vou fazer push da minha parte hoje, daí você rebaseia amanhã antes de continuar a sua”). Muitas vezes, conflitos são evitados compartilhando planos.
- **Atualizar a branch frequentemente:** já falamos, mas reforçando: se você está em uma branch de longa duração, traga as mudanças da main para ela regularmente (via merge da main ou rebase). Isso resolve conflitos em doses pequenas contínuas, em vez de um possivelmente grande no fim. Além disso, ao detectar conflito cedo, você pode até avisar o(a) outro(a) dev: “Vi que nossas mudanças conflitam; vou ajustar aqui mantendo a sua implementação daquela função, tudo bem?”.
- **Ferramentas de diff e merge:** o Git por si só avisa conflitos, mas não tenta resolvê-los (exceto casos triviais). Para ajudar na resolução manual, há ferramentas visuais de três vias (que mostram seu branch, o branch alvo e o ancestral comum). Exemplos: KDiff3, Meld, VS Code merge tool etc. Vale a pena familiarizar-se com alguma, pois ver as diferenças lado a lado acelera a compreensão do conflito. No entanto, muitas resoluções são simples e podem ser feitas no próprio editor, como demonstrado no hands on com os marcadores. O GitHub Web até permite resolver conflitos pelo navegador em PRs, mas para conflitos complexos é melhor localmente.
- **Aceitar as duas mudanças (ou partes delas):** um conflito não implica que uma mudança vai “ganhar” e a outra “perder” por completo. Às vezes, as duas edições podem e deveriam coexistir – você terá que editar para

combinar as ideias. Por exemplo, se um commit traduz uma interface para português e outro commit, paralelo, corrige um cálculo na mesma linha de texto, o merge conflitará porque um alterou texto e outro alterou números na mesma linha; a resolução correta seria incorporar ambos: texto traduzido e cálculo corrigido. Portanto, pense no contexto do conflito e certifique-se de que a versão final não perca nada importante. Teste o código após resolver para garantir que a funcionalidade de ambos os lados do conflito continua presente.

- **Evitar conflitos estruturalmente:** alguns padrões de projeto e organização minimizam a incidência de conflitos. Por exemplo, manter funções pequenas e focadas diminui a chance de dois devs editarem exatamente a mesma função simultaneamente. Modularizar o código em vários arquivos também ajuda (é mais provável dois devs trabalharem em arquivos distintos do que no mesmo).

Em notebooks Jupyter, conflitos são notórios porque notebooks são difíceis de mesclar (linhas enormes JSON) – por isso, uma dica é evitar que dois trabalhem no mesmo notebook; quebrem análises em notebooks separados ou transformem notebooks críticos em scripts .py quando for trabalho colaborativo (muitos times de dados convertem notebooks em código modular para facilitar o versionamento). Em geral, dividir bem as tarefas e modularizar o código é bom tanto para organização quanto para evitar conflitos.

Se apesar de tudo um conflito complexo ocorrer e você não tiver certeza de como resolvê-lo, envolva o time: dois pares de olhos podem pensar melhor que um. Use o próprio PR para discutir: “Pessoal, conflitou aqui, acho que minha versão do algoritmo e a do João diferem, qual mantemos?”. Essa colaboração na resolução também é aprendizado.

Sobre as boas práticas de commits em equipe, já tocamos neste assunto, mas vale enfatizar alguns pontos específicos de contexto multi-dev:

Commits pequenos, novamente: em equipe, commits isolados facilitam git blame (saber quem mudou o quê) e revert cirúrgico se precisar. Além disso, numa code review, se seu PR tiver vários commits, o revisor pode analisá-los um a um (no

GitHub dá para ver diffs commit por commit). Se cada commit tem uma lógica, isso ajuda na revisão incremental.

Padronização de mensagens e tags: muitas equipes adotam prefixos nas mensagens de commit para indicar tipo de mudança, por exemplo: feat, fix, docs, refactor, test, seguindo a convenção de Conventional Commits. Isso não apenas organiza o histórico, mas também permite geração automática de changelogs e version bump (ferramentas leem commits para criar notas de versão). Em ciência de dados, talvez não precise ser tão formal, mas vale manter consistência. Outra prática comum: incluir ID da tarefa no commit ou PR (ex.: “feat: adicionar conversão de temperatura (issue #45)”). Assim, no histórico fica claro o vínculo com o sistema de gerenciamento de projetos.

Não commitar arquivos desnecessários: parece óbvio, mas em equipe acontece de alguém acidentalmente commitar um arquivo grande ou confidencial. Mantenha um .gitignore atualizado para não versionar, por exemplo, dados brutos, chaves API, arquivos temporários, resultados intermediários pesados etc. Um .gitignore bem feito evita dores de cabeça e merge requests pedindo “remova por favor aquele arquivo de 500MB do commit”. No DEV Community um autor exemplificou .gitignore excluindo node_modules, arquivos de log, IDE configs etc., para manter o repositório limpo. Em projetos de dados, coloque no .gitignore coisas como: .ipynb_checkpoints/ (checkpoints do Jupyter), diretórios de output, arquivos .csv enormes e assim por diante.

Segurança e revisões obrigatórias: em ambientes corporativos, configure proteções no GitHub: branches protegidas que exigem X revisores aprovando antes do merge ou que impedem push direto na main (só via PR). Isso força as boas práticas. Também habilite verificações como “não permitir merge com testes falhando”. Tais configurações criam uma cultura em que todos passam pelo fluxo correto. No contexto de dados sensíveis, também há ferramentas de secret scanning que evitam que senhas/token vazem em commits – o GitHub oferece isso nativamente em repositórios públicos e pode ser habilitado nos privados.

Continuous Integration/Continuous Delivery (CI/CD): vale reforçar essa peça, pois colabora ativamente com a qualidade do trabalho em equipe. CI é o processo de executar builds e testes automatizados a cada mudança integrada. Já CD (ou deployment contínuo) trata de implantar automaticamente as mudanças

aprovadas. Em projetos de software, o CI/CD é padrão; em projetos de ciência de dados, começa a ser – pense em pipelines de dados, aplicações de Machine Learning em produção etc., em que toda alteração de código deveria passar por testes de desempenho ou de acurácia antes de ser aceita.

Por exemplo, se sua equipe mantém um modelo de previsão em produção, vocês podem ter testes unitários (verificando o comportamento de funções) e também testes de performance (assegurando que o modelo roda em X segundos e atinge pelo menos a mesma métrica de acurácia da versão anterior). O CI integraria isso no fluxo: toda vez que alguém abrir um PR com alterações no modelo ou no código ETL, o pipeline de teste rodaria e avisaria no PR se algo piorou.

Ferramentas como GitHub Actions facilitam configurar CI para repositórios de data science – há ações prontas para rodar notebooks e comparar outputs, por exemplo. A introdução de CI/CD nos workflows de equipes de dados é uma tendência forte (MLOps) que aproxima ainda mais o trabalho do(a) cientista de dados das práticas de engenharia de software tradicionais, aumentando confiabilidade e permitindo iterações rápidas com segurança.

Fechando esta seção, podemos afirmar: colaborar com Git/GitHub não é apenas usar comandos, mas sim adotar uma cultura e um conjunto de práticas. Envolve escrever mensagens claras, revisar colegas, planejar integrações frequentes, cuidar da qualidade com testes e automações e aprender com os erros (inclusive usando post-mortems quando um bug passou despercebido pelo processo, para melhorar ele).

Times que dominam essas práticas conseguem escalar projetos com muito mais desenvolvedores(as) sem perder o controle. Não por acaso, projetos enormes como o Linux Kernel (com milhares de contribuidores e empresas envolvidas) usam Git como backbone – a cada versão do kernel (que sai ~a cada 9-10 semanas), tipicamente mais de 1.500 desenvolvedores(as) contribuem e ~10.000 commits são mesclados e só é possível coordenar isso porque existe uma hierarquia de revisões e merges extremamente bem organizada (maintainers, pull requests por subsistemas, etc.).

Esse é um exemplo extremo, mas inspirador: mostra que, com processos e ferramentas adequados, mesmo algo de imensa complexidade pode evoluir

rapidamente sem virar bagunça. Trazendo para nosso dia a dia: seja um projeto de 3 pessoas ou de 30, aplicar princípios de colaboração com versionamento consistente faz toda a diferença na qualidade final e na tranquilidade do trabalho.

EMENDAS

MERCADO, CASES E TENDÊNCIAS

Errors de versão: Knight Capital Group (2012)

Caso Henrico Dolfing (2019) – Bug enviou US\$ 7 bilhões em ordens: sem testes/regressão e sem controle de versão rigoroso [\[henricodolfing.com\]](https://henricodolfing.com)

Quellit.ai (2025) – Deploy com flag de teste ativo, sem revisão dupla, causou prejuízo de US\$ 440 milhões [\[quellit.ai\]](https://quellit.ai), [\[alexponomarev.me\]](https://alexponomarev.me)

LinkedIn-Ankit Ghosh (2025) – Ativação de flag legado e falta de UAT/regressão resultaram em perdas rápidas [\[linkedin.com\]](https://linkedin.com)

Benefícios do Git/GitHub em ciência de dados: Pew Research Center

GitHub oficial “pewresearch” – Repositórios ativos (pewmethods, pewanalytics...), commits recentes [\[github.com\]](https://github.com)

arXiv / PLoS Biology (2025) – Fluxo completo: issues, project boards, containers para reprodutibilidade em biologia [\[arxiv.org\]](https://arxiv.org), [\[journals.plos.org\]](https://journals.plos.org)

SciSimple (2025) – GitHub como ferramenta de planejamento, documentação e ambiente em pesquisa experimental [\[scisimple.com\]](https://scisimple.com)

Tendências em branching: Trunk-Based vs Git Flow

Mergify Blog (2025) – TBD com merge queues favorece integração rápida; Gitflow é mais custoso [\[mergify.com\]](https://mergify.com)

arXiv @ UFPE (jul/2025) – Estudo com desenvolvedores brasileiros analisa contextos ideais (TBD para equipes menores/agilidade) [\[arxiv.org\]](https://arxiv.org)

LinkedIn-Padam Tripathi (out/2025) – GitFlow para releases regulados; TBD para pipelines ágeis de dados [\[linkedin.com\]](https://linkedin.com)

Aviator (jul/2025) – Detalha requisitos culturais e CI/CD para adoção efetiva de TBD com merge queues [\[aviator.co\]](https://aviator.co)

GitHub como rede + funcionalidades atuais

ElectrolQ (set/2025) – +150 milhões de devs, >1 bilhão de repositórios, GitHub Copilot com >20 milhões de usuários [\[electroiQ.com\]](https://electroiQ.com)

Coinlaw (jun/2025) – >100 mi de devs ativos, >420 mi repositórios, +70 000 projetos generative AI referenciados [\[coinlaw.io\]](https://coinlaw.io)

GitHub Blog (out/2025) – 180 mi+ devs, +1/repo criado por segundo, +518 milhões de PRs; TypeScript tomou liderança [\[github.blog\]](https://github.blog)

SQ Magazine (out/2025) – ações e pipelines CI/CD em uso corporativo; Copilot com agentes automatizados entre equipes [\[sqmagazine.co.uk\]](https://sqmagazine.co.uk)

Versionamento de dados e ML: DVC e MLflow

Dev.to (fev/2025) – Exemplo prático DVC + MLflow para versionar datasets + modelos em Python [\[dev.to\]](https://dev.to)

ML Journey (jun/2025) – Comparativo DVC × MLflow × W&B, benefícios de rastreabilidade, experimentação [\[mljourney.com\]](https://mljourney.com)

Site oficial DVC (2025) – Integração com Git, configuração eficiente para controle de dados em MLOps [\[dvc.org\]](https://dvc.org)

Codezup (mai/2025) – Boas práticas MLOps combinando DVC, MLflow e BentoML, com alvos de reprodutibilidade [\[codezup.com\]](https://codezup.com)

LakeFS Blog (dez/2024) – Versionamento robusto de dados e modelos via MLflow, reguladorizado por model registry [\[lakefs.io\]](https://lakefs.io)

Materiais gratuitos sobre Git/GitHub

Vídeos tutoriais:

freeCodeCamp.org – “Git and GitHub for Beginners” (1h08, ~4,8 Mi visualizações) [\[youtube.com\]](https://youtube.com)

freeCodeCamp.org – Curso “Learn Git – Full Course” (3h43, ~1,3 Mi visualizações) [\[youtube.com\]](https://youtube.com)

Kevin Stratvert – Tutorial prático 46 min com GitHub (PRs, branches, colab) [\[youtube.com\]](https://youtube.com)

Shahid Naeem (2025) – Playlist Git/GitHub (Hindi/Urdu, com 11 vídeos) [\[youtube.com\]](https://youtube.com)

AnalyticsInsight (jun/2025) – Canais gratuitos no YouTube (FreeCodeCamp, Traversy Media) [\[analyticsinsight.net\]](https://analyticsinsight.net)

Artigos, recursos:

GitHub Docs – “Git and GitHub learning resources” (GitHub Skills, Pro Git, Git branching visual) [\[docs.github.com\]](https://docs.github.com)

dev.to (jan/2023, atualizado 2025) – Top 12 recursos gratuitos com tutoriais, guias e livros [\[dev.to\]](https://dev.to)

MLTut (set/2025) – Lista com 40 recursos: cursos Google, IBM, Meta, GitHub Actions ... [\[mltut.com\]](https://mltut.com)

git-scm.com – Livro “Pro Git” gratuito, vídeos, cheat-sheet de comandos gráficos [\[git-scm.com\]](https://git-scm.com)

O QUE VOCÊ VIU NESTA AULA?

Recapitulando os aprendizados: começamos entendendo por que controle de versão é crucial, refletindo sobre problemas comuns (perda de trabalho, dificuldade em reproduzir resultados) e como o Git oferece soluções eficazes para cientistas de dados. Vimos na prática (seção Hands On) como criar repositórios Git, salvar alterações incrementais do código Python e organizar nosso desenvolvimento em branches, sempre com commits atômicos e mensagens claras.

Depois, aprofundamos conceitos teóricos no Saiba Mais, cobrindo desde boas práticas (commits frequentes, mensagens padronizadas, uso de `.gitignore`) até detalhes técnicos (como reverter mudanças com segurança e garantir a integridade do histórico), conectando com fundamentos de computação e com a realidade de projetos de ciência de dados colaborativos.

Exploramos também casos reais, reforçando lições importantes: um processo de versionamento mal conduzido pode levar a um fracasso estrondoso (Knight Capital), enquanto aderir a essas práticas traz benefícios de qualidade e colaboração reconhecidos (Pew Research Center, entre outros). Por fim, destacamos tendências de mercado – a onipresença do Git/GitHub, a evolução de workflows (Git Flow vs. trunk-based), a integração do versionamento ao ciclo de vida de ciência de dados – e recursos atuais para continuar aprendendo.

Em resumo, você deve sair desta aula com uma compreensão sólida de como usar Git no seu dia a dia de projetos, consciente das armadilhas a evitar e equipado(a) com técnicas para garantir que seu código esteja sempre versionado, documentado e seguro. No próximo documento, partiremos desse conhecimento para abordar colaboração em equipe, incluindo pull requests, code reviews e resolução de conflitos, aprofundando ainda mais sua habilidade em engenharia de software para ciência de dados.

Não deixe de praticar: que tal iniciar agora mesmo um repositório Git para algum projeto pessoal e aplicar essas lições? Assim, você consolida o que aprendeu e estará pronto(a) para os próximos desafios!

REFERÊNCIAS

CHACON, S.; STRAUB, B. **Pro Git**. 2. ed. Apress, 2014. Disponível em: <https://git-scm.com/book/en/v2>. Acesso em: 12 dez. 2025.

DANJOU, J. **Trunk-Based Development vs Gitflow**: Which Branching Model Actually Works? 2025. Disponível em: <https://mergify.com/blog/trunk-based-development-vs-gitflow-which-branching-model-actually-works>. Acesso em: 12 dez. 2025.

NILSEN, P. **Version Control Best Practices – Mastering Git, Branching Strategies and Collaborative Workflows**. DEV Community, 17 set. 2024. Disponível em: <https://dev.to/pellenilsen/version-control-best-practices-mastering-git-branching-strategies-and-collaborative-workflows-5gal>. Acesso em: 12 dez. 2025.

REMY, E. **How Pew Research Center uses git and GitHub for version control**. 2022. Disponível em: <https://www.pewresearch.org/decoded/2022/08/01/how-pew-research-center-uses-git-and-github-for-version-control/>. Acesso em: 12 dez. 2025.

GITHUB. **100 million developers and counting**. 2023. Disponível em: <https://github.blog/2023-01-25-100-million-developers/>. Acesso em: 12 dez. 2025.

MARTINS, L. **Git: Boas práticas para escrita das mensagens de commits**. 2023. Disponível em: <https://www.dio.me/articles/git-boas-praticas-para-escrita-das-mensagens-de-commits>. Acesso em: 12 dez. 2025.

PEW RESEARCH CENTER. **How we review code at Pew Research Center**. 2022. Disponível em: <https://www.pewresearch.org/decoded/2022/09/15/how-we-review-code-at-pew-research-center/>. Acesso em: 12 dez. 2025.

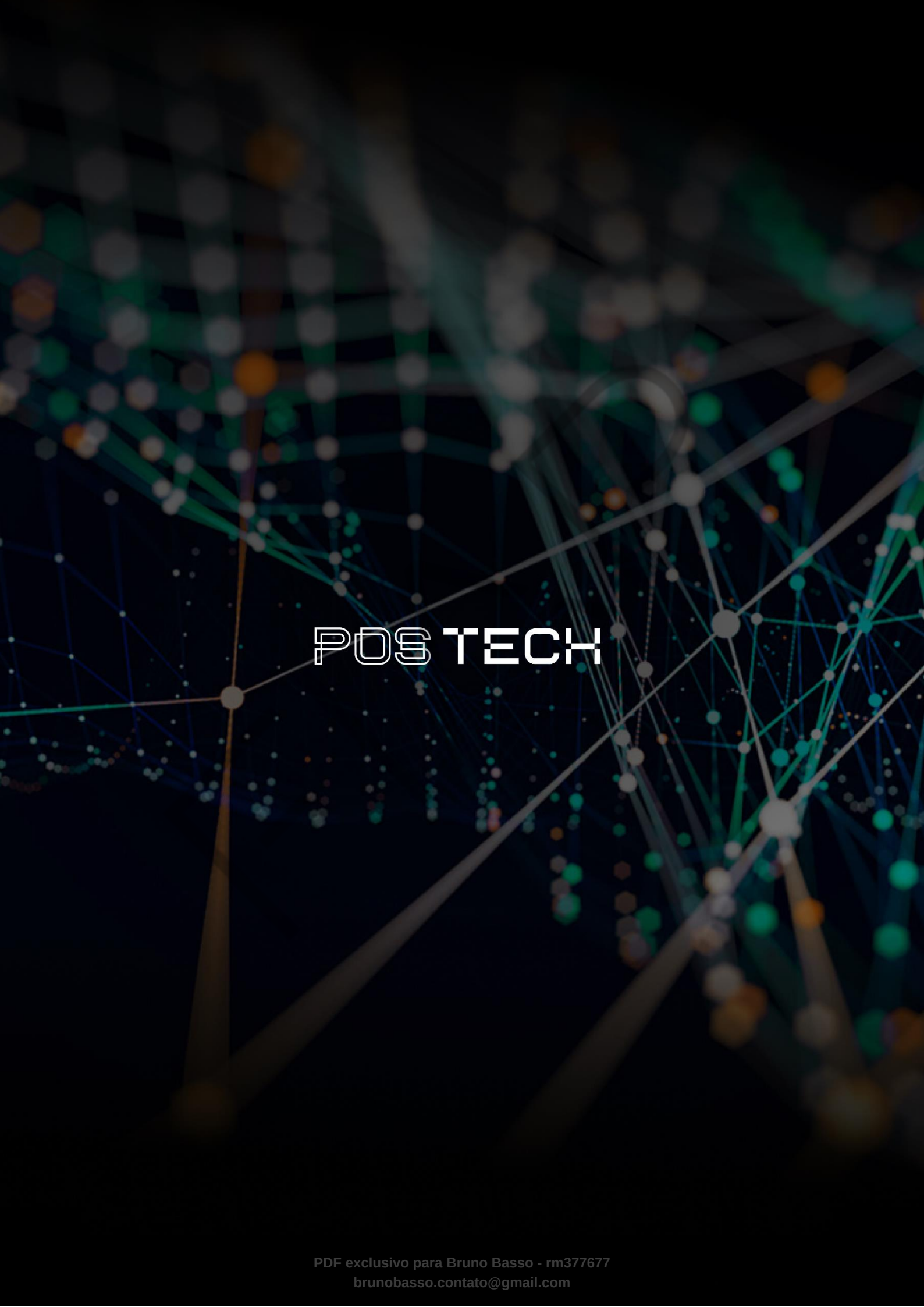
QEEDIO. **When Software Goes Unchecked**: 2025. Disponível em: <https://www.qeedio.com/posts-en/when-software-goes-unchecked-financial-giant-knight-capital-nearly-ruined>. Acesso em: 12 dez. 2025.

TANRIKULU, O. **Git Flow vs. Trunk-Based Development**: Choosing the Right Branching 2025. Disponível em: <https://medium.com/@ozlemtanrikulu/git-flow-vs-trunk-based-development-choosing-the-right-branching-strategy-38d9c2d23b78/>. Acesso em: 12 dez. 2025.

PALAVRAS-CHAVE

Colaboração em Desenvolvimento de Software. Controle de Versão Distribuído.
DevOps e MLOps.

EMENDAS



POSTECH